



בית ספר מקיף ז' הקריה אשדוד



Online Checkers

תיק פרויקט בהנדסת תוכנה

חלופה : תכנות אסינכרוני ומסדי נתונים

התלמיד : עידן ברשישט

ת.ז. : 332940972

מנחה : אולגה גנדלמן

תאריך : יוני 2026

1. מבוא ורקע לפרויקט	4
1.1. רקע והשראה לפרויקט	4
1.2. שם הפרויקט ותיאורו	4
1.3. קהל היעד	4
1.4. מטרות המערכת	4
1.5. תיאור כללי של המערכת	5
1.6. גבולות המערכת	5
1.7. סביבות פיתוח, שפות ושכבות	6
1.8. אתגרים מרכזיים בפרויקט	6
1.9. חידושים ומאפיינים מיוחדים	6
1.10. מדריך למשתמש	6
2. מענה למחווה על פי דרישות הבחינה	8
2.1. דרישות חובה (סעיפים 1 עד 8)	8
2.2. הרחבות לסעיף 9 (יכולות בסיס)	8
2.3. הרחבות לסעיף 10 (יכולות מתקדמות)	8
2.4. סיכום המענה למחווה	8
3. ניתוח המערכת	9
3.1. מסמך ייזום	9
3.2. מצב קיים מול מצב עתידי	9
3.3. בעלי תפקידים במערכת	9
3.4. תרשימי שימוש (Use Cases)	9
3.5. עץ תהליכים (Process Tree)	9
3.6. תרשים קשרי גומלין (ERD)	10
4. בסיס הנתונים	11
4.1. בחירת מסד הנתונים	11
4.2. פירוט הטבלאות (מילון נתונים)	11
4.3. קשרי הגומלין (ERD)	11
4.4. דוגמאות לשאלות במערכת	11
5. ארכיטקטורת המערכת	13
5.1. תרשים שכבות (Layered Architecture)	13
5.2. מבנה הסולושן (Solution Structure)	13
5.3. תרשים פריסה (Deployment Diagram)	14
5.4. זרימת הנתונים בין הרכיבים	14
6. צד שרת	16
6.1. רכיבי השרת	16
6.2. שירות WCF ראשי (ICheckersService)	16
6.3. לוגיקה עסקית (GameManager ו-GameSession)	16
6.4. תכנות אסינכרוני (Asynchronous Programming)	16
6.5. שכבת גישה הנתונים (Data Access Layer)	17
6.6. אתר הניהול (ASP.NET Razor Pages)	17
6.7. תקשורת בזמן אמת (Polling)	17

7. צד לקוח.....	18
7.1 לקוח WPF Desktop	18
7.2 לקוח אתר הניהול (Web)	18
7.3 לקוח MAUI (Android ו-Windows)	18
8. אבטחה והגנה על נתונים	19
8.1 הצפנת סיסמאות עם PBKDF2	19
8.2 הגנה מ-SQL Injection	19
8.3 ניהול אימות מבוסס טוקן	19
8.4 אכיפת חוקים בצד השרת	20
8.5 הגנת Admin	20
9. תהליכים מרכזיים במערכת	21
9.1 תהליך התחברות	21
9.2 תהליך התאמת משחק (Matchmaking)	21
9.3 תהליך פתיחת משחק	21
9.4 תהליך ביצוע מהלך	21
9.5 אלגוריתם היריב הממוחשב (Bot)	22
9.6 תהליך הניתוק	22
10. בדיקות ותסריטי שימוש	23
10.1 בדיקות ידניות שביצעתי	23
10.2 בדיקות עומס	23
10.3 בדיקות אבטחה	23
11. מדריך למשתמש	24
11.1 איך מתחילים?	24
11.2 רישום משתמש חדש	24
11.3 התחברות	24
11.4 תפריט ראשי, מה ניתן לעשות?	24
11.5 ביצוע מהלכים	24
11.6 המשחק	25
11.7 רוצה לשחק מול חבר או מול הבוט?	25
12. סיכום אישי, רפלקציה	26
13. ביבליוגרפיה	27
13.1 ספרות ומדריכי לימוד	27
13.2 אתרי עזר טכניים	27
13.3 כלים וספריות צד שלישי	27
13.4 שימוש בבניה מלאכותית	27
14. נספחים, קוד מלא	28
14.1 מבנה הפרויקטים בסולושן	28
14.2 דוגמת קוד, מחלקת BaseDB (מחלקה אבסטרקטית)	28
14.3 דוגמת קוד, מנוע המשחק (יצירת מהלכים)	28
14.4 דוגמת קוד, הצפנת סיסמאות (PasswordHasher)	29
14.5 הערה לסיום	30

1. מבוא ורקע לפרויקט

1.1 רקע והשראה לפרויקט

הבחירה לפתח את Online Checkers נובעת מאהבה אישית למשחקי לוח אסטרטגיים, ובמיוחד למשחק הדמקה הקלאסי. הכללים פשוטים ללמידה, אך מתחתם מסתתר עומק טקטי אמיתי: כל מהלך משפיע על מבנה הלוח, אכילה כפויה משנה את מאזן הכוחות, והכתרת מלך הופכת כלי פשוט לעוצמתי. השילוב הזה יוצר חוויה הוגנת ולא צפויה גם אחרי משחקים רבים.

המטרה שלי בפרויקט הייתה לקחת את החוויה הקלאסית הזו ולהעביר אותה לעידן המודרני: לאפשר משחק רב-משתתפים בזמן אמת דרך הרשת בין שני שחקנים היושבים על מחשבים שונים, להוסיף יריב מבוסס מחשב לשחקן הבודד, לנהל מערכת דירוג Elo גלובלית, ולספק את כל זה דרך שלושה לקוחות נפרדים: דסקטופ (WPF), אתר ניהול בדפדפן (ASP.NET), ומובייל (MAUI), שכולם פונים לאותו שרת ולאותו מסד נתונים.

המערכת בנויה כארכיטקטורת שרת-לקוח מלאה: השרת אחראי על הלוגיקה, על שמירת הנתונים ועל סנכרון השחקנים, והלקוחות בכל שלוש הפלטפורמות הם תצוגה ובקרת מצב בלבד. גישה זו מבטיחה שכל חוקי המשחק נאכפים במקום אחד מרכזי ומאובטח.

1.2 שם הפרויקט ותיאורו

שם הפרויקט: Online Checkers, משחק דמקה רב-משתתפים בזמן אמת.

המערכת היא יישום שרת-לקוח מלא. כל שחקן נרשם, מתחבר ומתחיל קרב תורות על לוח 8x8 מול יריב אנושי או מול יריב מבוסס מחשב (בוט). הכלים ניצבים במצב פתיחה קבוע, וכל תור השחקן בוחר כלי ומבצע מהלך חוקי: תנועה אלכסונית פשוטה או אכילה. במהלך המשחק כל מהלך מתעדכן בזמן אמת אצל שני הצדדים, מתעדכן מצב הלוח, נשמרת היסטוריית המהלכים, ומתבצעות הצעות תיקון והתפטרות. בסיום נשמרות הסטטיסטיקות במסד הנתונים, מתעדכן דירוג ה-Elo, וניתן לצפות שוב במשחק שהסתיים. בנוסף קיים ממשק ניהול (Admin) לחסימת משתמשים ולצפייה בנתוני המערכת.

1.3 קהל היעד

המערכת פונה לכמה סוגי משתמשים שונים:

שחקנים פרטיים שאוהבים משחקי לוח אסטרטגיים, מכל גיל, ללא צורך בידע טכני.

שחקנים שמחפשים חוויה מהירה מול יריב חי, ישירות מהמחשב, הדפדפן או הטלפון.

שחקנים בודדים שרוצים לתרגל מול יריב ממוחשב מבלי להמתין ליריב אנושי.

שחקנים תחרותיים המעוניינים לטפס בטבלת המובילים הגלובלית באמצעות מערכת דירוג Elo.

מנהלי מערכת (Admin) האחראים על ניהול הקהילה, חסימת משתמשים ומעקב אחר נתוני השרת.

1.4 מטרות המערכת

מטרות על:

יצירת חוויית משחק רב-משתתפים בזמן אמת מול שרת מרכזי אחד.

אכיפת כל חוקי הדמקה בצד השרת בלבד, כך שאף שחקן אינו יכול לרמות.

הצגת אותה מערכת באמצעות שלושה לקוחות שונים (WPF, אתר ניהול ו-MAUI) המתחברים לאותו שרת ולאותו מסד נתונים.

מתן ממשק נקי, אחיד וברור בכל שלוש הפלטפורמות.

מערכת דירוג Elo גלובלית לדירוג השחקנים לפי תוצאותיהם.

מטרות נלוות:

הצפנת סיסמאות באמצעות PBKDF2 עם Salt אקראי למניעת דליפות מידע.
זיהוי אוטומטי של ניתוקי לקוח (קריסה או ניתוק רשת) ושחרור היריב מהמשחק.
יריב ממוחשב (בוט) לשחקן הבודד.
צפייה חוזרת (Replay) במשחקים שהסתיימו, מהלך אחר מהלך.
פאנל ניהול (Admin) המאפשר לצפות בכל המשתמשים, לחסום משתמשים ולראות נתוני שרת.

1.5 תיאור כללי של המערכת

המערכת בנויה לפי ארכיטקטורת שכבות נקייה, עם הפרדה מלאה בין שכבת התצוגה, שכבת הלוגיקה ושכבת הנתונים. בחירה זו מאפשרת לצרף לקוחות נוספים בעתיד מבלי לגעת בקוד השרת או במסד הנתונים, וכך אכן נוספו אתר הניהול ואפליקציית המובייל לאחר שכבר היה לקוח WPF פעיל.

השכבות העיקריות במערכת הן :

שכבת Model : מחלקות נתונים (User, Board, Piece, Square, Move, GameStateDto ועוד) המשותפות לשרת וללקוחות דרך WCF DataContract.

שכבת ViewDB : שכבת גישה למסד הנתונים, היורשת ממחלקה אבסטרקטית BaseDB המכילה את הפעולות הכלליות (Select, SaveChanges) עם הגנה מלאה מפני SQL Injection באמצעות פרמטרי OleDb.

שכבת BusinessLogic : לוגיקה עסקית, הכוללת את מנוע המשחק (CheckersEngine), ניהול המשחקים (GameManager), היריב הממוחשב (BotEngine), אימות (AuthService) והצפנת סיסמאות (PasswordHasher).

שכבת WCF Server : שירות WCF המתארח כתהליך עצמאי וחושף שתי נקודות קצה במקביל (net.tcp ו-basicHttp).

שכבת לקוח WPF : ממשק שולחני עשיר, מבוסס Views ו-ViewModels בתבנית MVVM, עם ניווט בין מסכים ועדכון מצב המשחק בזמן אמת.

שכבת אתר הניהול (ASP.NET Razor Pages) : אתר בדפדפן המספק למנהל לוח בקרה, ניהול משתמשים וצפייה במשחקים אחרונים.

שכבת לקוח MAUI : אפליקציה נטיבית לאנדרואיד ול-Windows, המתחברת לאותו שירות.

1.6 גבולות המערכת

בכל פרויקט תוכנה חשוב להגדיר במפורש מה כלול ומה אינו כלול, כדי לא לקרוס תחת עומס דרישות. הבחירות שלי היו :

המערכת כוללת :

שלושה לקוחות פעילים : WPF Desktop, אתר ניהול בדפדפן, ו-MAUI (Android ו-Windows).

שני סוגי משתמשים, שחקן רגיל ומנהל (Admin), כאשר ההבחנה נשמרת בשדה Role בטבלת המשתמשים.

משחק מול יריב ממוחשב ומול שחקן אחר.

אכיפת כל חוקי הדמקה בצד השרת, כולל אכילה כפויה, שרשור אכילות והכתרת מלך.

המערכת אינה כוללת :

הצפנת תקשורת ברמת התווך (TLS/HTTPS) ; הקריאות מתבצעות ב-net.tcp ו-HTTP בסביבה מקומית. הצפנה ניתנת להוספה בעתיד דרך הגדרת תעודה.

אינטגרציה עם רשתות חברתיות או התחברות דרך שירות חיצוני.

יריב ממוחשב מבוסס אלגוריתם חיפוש מתקדם ; הבוט הנוכחי בוחר מהלך חוקי בלבד, עם העדפה להכתרה.

1.7 סביבות פיתוח, שפות ושכבות

סביבת פיתוח: Microsoft Visual Studio.

שפות תכנות: #C עבור לוגיקת השרת והלקוחות, XAML עבור עיצוב ממשק המשתמש בדסקטופ ובמובייל, ו-HTML ו-Razor עבור אתר הניהול.

טכנולוגיות מרכזיות:

WPF (Windows Presentation Foundation) לצד הלקוח השולחני.

WCF (Windows Communication Foundation) לניהול התקשורת מול השרת המרכזי.

ASP.NET Core Razor Pages לאתר הניהול.

.NET MAUI לאפליקציית המובייל החוצה-פלטפורמות.

OleDb לעבודה מול בסיס נתונים Microsoft Access.

ארכיטקטורת שכבות: הפרויקט מחולק לשכבות Model (ייצוג הנתונים), ViewDB (גישה למסד הנתונים), BusinessLogic (לוגיקת המשחק והדירוג) והשירות (CheckersService).

1.8 אתגרים מרכזיים בפרויקט

במהלך הפיתוח התמודדתי עם כמה אתגרים מרכזיים. אני מציין כאן את החשובים שבהם, יחד עם הפתרון שמצאתי לכל אחד:

סנכרון מצב המשחק בין שני שחקנים: הצורך לוודא ששני הצדדים רואים תמיד את אותו מצב לוח, ושמהלך של אחד לא יידרס על-ידי מצב ישן של השני. הפתרון: השרת מחזיק עותק יחיד ואמין של מצב המשחק בזיכרון, וכל הלקוחות דוגמים אותו במרווחים קצרים. מהלך מאושר רק לאחר שעבר בדיקת חוקיות בשרת.

חישוב כל רצפי האכילה: על פי חוקי הדמקה, אכילה היא חובה, ויש להמשיך ולאכול ברצף כל עוד אפשר. הפתרון: מימוש חיפוש לעומק (DFS) רקורסיבי במנוע המשחק, המסיר את הכלי שנאכל בכל שלב וממשיך לחפש אכילות נוספות עד שלא נותרות אפשרויות.

זיהוי ניתוקי לקוח: לקוח שקורס אינו מספיק להודיע לשרת שעזב, ולכן היריב עלול להישאר תקוע. הפתרון: טיימר בשרת (StaleSweep) הסורק את המשחקים הפעילים ומסיים משחק שבו שחקן לא דיווח על פעילות במשך פרק זמן מוגדר, תוך עדכון התוצאה והטלת קנס דירוג.

1.9 חידושים ומאפיינים מיוחדים

יש בפרויקט כמה מאפיינים שאינם נפוצים בפרויקטי תיכון:

אכיפת כל חוקי המשחק בצד השרת בלבד (Anti-Cheat): הלקוח אינו יכול לזייף אכילה, לדלג על אכילת חובה או לבצע מהלך לא חוקי.

שלושה לקוחות נפרדים (WPF, אתר ניהול ו-MAUI) המתחברים לאותו שרת ולאותו מסד נתונים דרך חוזה DataContract משותף.

מנוע משחק מלא המטפל בשרשור אכילות באמצעות חיפוש רקורסיבי לעומק.

מערכת דירוג Elo המתעדכנת אוטומטית בסיום כל משחק, עם רצפה שמונעת צניחת דירוג לאפס.

צפייה חוזרת (Replay) במשחקים שהסתיימו, המבוססת על שמירת מצב הלוח לאחר כל מהלך.

זיהוי אוטומטי של ניתוקים באמצעות טיימר ניקוי בשרת.

הצפנת סיסמאות חזקה באמצעות PBKDF2-SHA256 עם Salt אקראי והשוואה בזמן קבוע.

1.10 מדריך למשתמש

תרשים דפים:

<תמונה : תרשים זרימת המסכים : מסך פתיחה – התחברות/רישום – לובי – לוח משחק – מסך סיום>
שחקן מתחיל את כניסתו למשחק ממסך הפתיחה. אם כבר יש לו משתמש, הוא יכול להזין את פרטיו ולהיכנס ישירות.

<תמונה : מסך ההתחברות>

אם אין לו משתמש, הוא עובר למסך הרישום, ממלא את פרטיו ונכנס למשחק.

<תמונה : מסך הרישום>

לאחר ההתחברות מגיע השחקן ללובי, המסך המרכזי של המערכת, שבו הוא רואה את הדירוג שלו, את רשימת השחקנים המחוברים ואת היסטוריית המשחקים, ויכול להתחיל חיפוש משחק.

<תמונה : מסך הלובי הראשי>

השחקן יכול לבחור לשחק משחק מדורג מול שחקן אחר, או לשחק מול הבוט.

<תמונה : לוח המשחק, עם הכלים של שני הצדדים ושמות השחקנים>

במהלך המשחק השחקן בוחר כלי, והמהלכים החוקיים שלו מודגשים. אם קיימת אכילת חובה, הוא מחויב לבצעה. בכל רגע ניתן להציע תיקו או להתפטר.

<תמונה : מסך הצעת תיקו או אישור התפטרות>

בסיום המשחק מוצג מסך תוצאה המעדכן את הדירוג, ולאחריו חוזר השחקן ללובי.

<תמונה : מסך סיום המשחק עם עדכון הדירוג>

השחקן יכול לצפות בטבלת המובילים הגלובלית של כל השחקנים לפי דירוג.

<תמונה : טבלת המובילים>

כן הוא יכול לראות את היסטוריית המשחקים שלו ולצפות שוב בכל אחד מהם.

<תמונה : מסך היסטוריית המשחקים והצפייה החוזרת>

בדף הפרופיל ניתן לעדכן דוא"ל, סיסמה ותמונת פרופיל, ולצאת מהמשתמש.

<תמונה : דף הפרופיל>

2. מענה למחווון על פי דרישות הבחינה

פרק זה מציג את המענה לדרישות המחווון של חלופת "שירותי אינטרנט, תכנות אסינכרוני ומסדי נתונים". המענה מוצג בשקיפות מלאה: לכל סעיף כתוב מה הפרויקט מספק, ובמקום שבו המענה חלקי הדבר מצוין במפורש.

2.1 דרישות חובה (סעיפים 1 עד 8)

מסד נתונים מנורמל: המערכת משתמשת ב-Microsoft Access עם שלוש טבלאות (Users, Games, Moves), הכוללות מפתחות ראשיים ומפתחות זרים.

פעולות CRUD מלאות: המערכת מבצעת INSERT (רישום משתמש, שמירת משחק ומהלך), SELECT (התחברות, טבלת מובילים, היסטוריה), UPDATE (עדכון סטטיסטיקות ודירוג) ו-DELETE (מחיקת משתמש בידי מנהל).

שאלות מאובטחות: כל השאלות משתמשות בפרמטרים של OleDb להגנה מפני SQL Injection.

ארכיטקטורת שרת-לקוח: שרת WCF מרכזי המשרת שלושה לקוחות נפרדים.

ממשק משתמש גרפי: ממשקים בנויים ב-WPF, ב-Razor וב-MAUI.

ניהול משתמשים והרשאות: רישום, התחברות, והפרדה בין שחקן רגיל למנהל.

2.2 הרחבות לסעיף 9 (יכולות בסיס)

ירושה ופולימורפיזם: כל מחלקות הנתונים יורשות ממחלקת הבסיס Base, וכל מחלקות הגישה לנתונים יורשות מ-BaseDB ומממשות פעולות מופשטות בדרכן.

אירועים ו-Delegates: לקוח ה-WPF משתמש באירועים מותאמים (RoutedEvent) ובפקודות (RelayCommand) לקישור בין הממשק ללוגיקה.

הצפנה: סיסמאות המשתמשים מוצפנות באמצעות PBKDF2-SHA256.

2.3 הרחבות לסעיף 10 (יכולות מתקדמות)

תכנות אסינכרוני: שכבת הגישה לנתונים והקריאות לשרת מבוצעות באופן אסינכרוני (Async/Await) כדי שהממשק לא ייתקע.

הגנה מפני SQL Injection: כל שאלתה מבוצעת עם פרמטרים בלבד.

אכיפת לוגיקה בצד השרת: כל חוקי המשחק מחושבים ונאכפים בשרת, מה שמונע מהלכים לא חוקיים מצד הלקוח.

2.4 סיכום המענה למחווון

שמונה דרישות החובה מקבלות מענה מלא, ובנוסף נוספו הרחבות בסעיף 9 (ירושה, אירועים, הצפנה) ובסעיף 10 (אסינכרוניות, הגנה מפני SQL Injection, אכיפת חוקים בצד השרת). החוזק היחסי של הפרויקט מתבטא בכך שאותו שרת משרת שלושה לקוחות שונים, ובכך שכל חוקי המשחק נאכפים אך ורק בשרת.

3. ניתוח המערכת

3.1 מסמך ייזום

שם הפרויקט: Online Checkers, משחק דמקה רב-משתתפים.

בעיה קיימת: משחק הדמקה הקלאסי מצריך שני אנשים היושבים פיזית באותו מקום ולוח אחד. אין דרך נוחה לשחק נגד חבר שגר רחוק, ואין מעקב אחר היסטוריית המשחקים, הדירוג או הסטטיסטיקות של השחקן.

פתרון מוצע: מערכת שרת-לקוח המאפשרת לשני שחקנים לשחק דמקה בזמן אמת מכל מקום, עם יריב ממוחשב לשחקן הבודד, מערכת רישום והתחברות מאובטחת, סטטיסטיקות אישיות, טבלת מובילים גלובלית, צפייה חוזרת במשחקים, ניהול ניתוקים אוטומטי, ולקוחות לכמה פלטפורמות.

ערך עסקי:

משחק רב-משתתפים מהיר וממוקד מכל מכשיר.

יריב ממוחשב שמאפשר תרגול גם ללא יריב אנושי.

שקיפות סטטיסטית: כל שחקן יודע כמה ניצח, כמה הפסיד ומה דירוגו.

תחרותיות גלובלית באמצעות טבלת המובילים.

גמישות פלטפורמה: משחק מהמחשב (WPF), מהדפדפן או מהטלפון (MAUI).

3.2 מצב קיים מול מצב עתידי

מצב קיים: משחק דמקה פיזי או יישומים מקומיים בלבד, ללא ניהול משתמשים, ללא דירוג וללא שמירת היסטוריה.

מצב עתידי (המערכת שלי): מערכת מקוונת מלאה עם חשבונות משתמש, דירוג Elo, היסטוריית משחקים, צפייה חוזרת, יריב ממוחשב, ניהול מנהלים, ושלושה לקוחות מעל שרת מרכזי אחד.

3.3 בעלי תפקידים במערכת

שחקן אורח: יכול לשחק מול הבוט ללא הרשמה.

שחקן רשום: נרשם, מתחבר, משחק מול שחקנים אחרים ומשפיע על דירוגו, וצופה בהיסטוריה ובטבלת המובילים.

מנהל (Admin): כל יכולות השחקן הרשום, ובנוסף צפייה בכל המשתמשים, חסימת משתמשים וצפייה בנתוני השרת.

השרת: שחקן לוגי שמנהל את כל המשחקים, אוסף את החוקים ושומר את הנתונים.

3.4 תרשימי שימוש (Use Cases)

<תמונה: תרשים Use Case המציג את הפעולות של שחקן רשום, אורח ומנהל מול המערכת>

3.5 עץ תהליכים (Process Tree)

עץ התהליכים מתאר את האפשרויות שיש לשחקן בכל רגע, לפי מצב המערכת. מהשורש (Online Checkers) מתפצלים כמה ענפים עיקריים:

שורש: Online Checkers.

ענף אימות וזיהוי: רישום משתמש חדש, התחברות, יציאה.

ענף משחק מול הבוט: התחלת משחק, ביצוע מהלכים, סיום, חזרה ללובי.

ענף משחק רב-משתתפים: כניסה לתור התאמה, המתנה ליריב, משחק, סיום, חזרה ללובי.

ענף צפייה בנתונים: טבלת מובילים, היסטוריית משחקים, צפייה חוזרת, פרופיל.

ענף ניהול (Admin בלבד) : רשימת משתמשים, חסימת משתמש, צפייה בנתוני השרת.

3.6 תרשים קשרי גומלין (ERD)

מבנה הנתונים מורכב משלוש טבלאות. הקשרים ביניהן :

Users ו-Games : קשר 1 ל-N. כל משחק מקשר בין שני משתמשים (WhitePlayerId ו-BlackPlayerId), וכל משתמש יכול להופיע במשחקים רבים.

Moves ו-Games : קשר 1 ל-N. לכל משחק שייכים מהלכים רבים, וכל מהלך שייך למשחק אחד.

<תמונה : תרשים ERD של שלוש הטבלאות (Users, Games, Moves) והקשרים ביניהן>

4. בסיס הנתונים

4.1 בחירת מסד הנתונים

מסד הנתונים של Online Checkers ממומש ב-Microsoft Access (קובץ .accdb), עם גישה דרך OleDb בלבד. הבחירה נובעת מכמה סיבות:

פשטות פיתוח: אין צורך בשרת מסד נתונים נפרד שיש להתקין ולהגדיר. הקובץ הוא קובץ יחיד היושב במחשב, וזה מספיק לפרויקט בקנה מידה זה.

עמידה בדרישות החלופה: המחווה מצפה למסד נתונים מנומל עם פעולות SELECT, INSERT, UPDATE ו-DELETE ופרמטרים מאובטחים, ו-Access מספק את כל אלה דרך OleDb.

ניתן ליצור שאילתות, לקבוע מפתחות זרים ולהשתמש בכל פעולות ה-SQL הסטנדרטיות.

4.2 פירוט הטבלאות (מילון נתונים)

מסד הנתונים מכיל שלוש טבלאות, וכל אחת מתעדת היבט אחר של המערכת:

1. טבלת Users, טבלת המשתמשים.

מאחסנת את פרטי המשתמשים הרשומים. זוהי הטבלה המשמשת לאימות, ואליה מתייחסות שאר הטבלאות.

שדות מרכזיים: ID (מפתח ראשי), Username (שם משתמש ייחודי), PasswordHash (תמצית הסיסמה המוצפנת), Email, Role (שחקן או מנהל), Rating (דירוג Elo), Wins, Losses, Draws (סטטיסטיקה), IsBanned (האם חסום), ProfilePicture (תמונת פרופיל בינארית).

<תמונה: צילום מסך של מבנה טבלת Users ב-Access>

2. טבלת Games, טבלת המשחקים.

בכל סיום משחק (מול שחקן אחר או מול הבוט) נוסף רקורד חדש. הטבלה משמשת גם כטבלת קישור בין שני שחקנים.

שדות מרכזיים: ID (מפתח ראשי), WhitePlayerId ו-BlackPlayerId (מפתחות זרים לטבלת המשתמשים), StartedAt ו-EndedAt (זמני התחלה וסיום), Status (מצב המשחק), EndReason (סיבת הסיום), WinnerId (המנצח), FinalBoard (מצב הלוח הסופי), ו-MoveCount (מספר המהלכים).

<תמונה: צילום מסך של מבנה טבלת Games ב-Access>

3. טבלת Moves, טבלת המהלכים.

בכל מהלך במשחק נוסף רקורד חדש. זה מאפשר צפייה חוזרת (Replay) במשחק שלם.

שדות מרכזיים: ID (מפתח ראשי), GameId (מפתח זר לטבלת המשחקים), MoveNumber (מספר המהלך), MoverColor (צבע השחקן), FromFile ו-FromRank ו-ToFile ו-ToRank (קואורדינטות המקור והיעד), PathSerialized (מסלול האכילה), IsCapture (האם אכילה), CapturedCount (כמה כלים נאכלו), BecameKing (האם בוצעה הכתרה), Notation (סימון המהלך), ו-BoardAfter (מצב הלוח לאחר המהלך).

<תמונה: צילום מסך של מבנה טבלת Moves ב-Access>

4.3 קשרי הגומלין (ERD)

הקשרים בין הטבלאות בנויים סביב מזהי המשתמשים והמשחקים:

Users ו-Games: קשר 1 ל-N. כל משחק מקשר בין שני שחקנים (לבן ושחור), וכל שחקן יכול לשחק משחקים רבים. למעשה טבלת Games מתפקדת כטבלת קישור המייצרת קשר רבים-לרבים בין השחקנים.

Moves ו-Games: קשר 1 ל-N. לכל משחק שייכים מהלכים רבים, וכל מהלך שייך למשחק יחיד דרך GameId.

4.4 דוגמאות לשאילתות במערכת

להלן דוגמאות לשאילתות אופייניות במערכת. כולן מוגנות מפני SQL Injection באמצעות פרמטרים (סימן ?).

שאילתת SELECT עם פרמטר (התחברות), נקראת בכל פעם שמשתמש מתחבר :

```
SELECT * FROM Users WHERE Username = ?
```

שאילתת INSERT (רישום משתמש חדש) :

```
INSERT INTO Users (Username, PasswordHash, Email, Role, Rating)
VALUES (?, ?, ?, ?, ?)
```

שאילתת ORDER BY עם TOP (טבלת מובילים), מחזירה את חמישים השחקנים המובילים לפי דירוג :

```
SELECT TOP 50 Username, Wins, Losses, Draws, Rating
FROM Users
ORDER BY Rating DESC, Wins DESC
```

שאילתת UPDATE (עדכון סטטיסטיקות ודירוג בסיום משחק) :

```
UPDATE Users
SET Wins = Wins + 1, Rating = ?
WHERE ID = ?
```

שאילתת SELECT עם חיבור (היסטוריית משחקים), מחזירה את כל המשחקים של השחקן עם שמות שני היריבים :

```
SELECT g.ID, g.StartedAt, g.WinnerId, g.Status,
       w.Username AS WhiteName, b.Username AS BlackName
FROM (Games AS g
      INNER JOIN Users AS w ON g.WhitePlayerId = w.ID)
      INNER JOIN Users AS b ON g.BlackPlayerId = b.ID
WHERE g.WhitePlayerId = ? OR g.BlackPlayerId = ?
ORDER BY g.StartedAt DESC
```

שאילתת SELECT של מהלכים (לצורך צפייה חוזרת) :

```
SELECT MoveNumber, MoverColor, Notation, BoardAfter
FROM Moves
WHERE GameId = ?
ORDER BY MoveNumber
```

שאילתת DELETE (מחיקת משתמש בידי מנהל) :

```
DELETE FROM Users WHERE ID = ?
```

5. ארכיטקטורת המערכת

5.1 תרשים שכבות (Layered Architecture)

המערכת בנויה לפי עיקרון של הפרדה ברורה בין שכבות. כל שכבה אחראית על דבר אחד מוגדר, וכל שכבה גבוהה משתמשת רק בשכבה שמתחתיה. החלוקה מבטיחה שאם מוסיפים לקוח חדש (כפי שנעשה כשנוספו אתר הניהול ו-MAUI), אין צורך לשנות דבר בשכבות התחתונות.

<תמונה: תרשים שכבות המערכת, מלמעלה למטה>

מלמעלה למטה:

שכבת תצוגה: שלושה לקוחות, WPF Desktop, אתר ניהול בדפדפן, ו-MAUI.

שכבת תקשורת: WCF בפרוטוקול net.tcp ו-basicHttp.

שכבת שירות: CheckersService המממש את ICheckersService.

שכבת לוגיקה עסקית: AuthService, BotEngine, GameSession, GameManager, CheckersEngine, PasswordHasher.

שכבת גישה נתונים (ViewDB): UsersDB, GamesDB, MovesDB, כולן יורשות מ-BaseDB.

שכבת Model: User, Board, Piece, Square, Move, ו-DTO שונים.

שכבת אחסון: קובץ Microsoft Access בשם ArchiveData3.accdb.

5.2 מבנה הסולושן (Solution Structure)

הקוד מאורגן בכמה פרויקטים בתוך סולושן אחד, ובנוסף פרויקטי לקוח נפרדים. להלן המבנה:

```
projectFinal/
|
+-- server/                                <- WCF שרת
|
|   +-- Program.cs                         <- (Console Host) אירוח השירות
|   +-- CheckersService.cs                 <- מימוש השירות
|   +-- ICheckersService.cs                <- חוזה השירות
|   +-- Bootstrap.cs                       <- אתחול מסד הנתונים וזריעת מנהל
|   +-- App.config                         <- הגדרות נקודות הקצה
|   +-- Model/                             <- מחלקות הנתונים
|       +-- Base, User, Board, Piece, Square, Move, Enums, DTOs
|   +-- BusinessLogic/
|       +-- CheckersEngine.cs              <- מנוע חוקי הדמקה
|       +-- GameManager.cs                 <- Elo-ניהול משחקים, התאמה ו
|       +-- GameSession.cs                 <- מצב של משחק יחיד
|       +-- BotEngine.cs                   <- היריב הממוחשב
|       +-- AuthService.cs                 <- אימות
|       +-- PasswordHasher.cs              <- הצפנת סיסמאות (PBKDF2)
|   +-- ViewDB/                           <- שכבת גישה נתונים
```

```

|      +-- BaseDB.cs          <- מחלקה אבסטרקטית
|
|      +-- UsersDB.cs, GamesDB.cs, MovesDB.cs
|
|      +-- DatabaseInitializer.cs, schema.sql
|
+-- client/                  <- WPF Desktop (MVVM) לקוח
|
|  +-- Views/                <- מסכים (Login, Lobby, Game, ...)
|
|  +-- ViewModels/           <- לוגיקת המסכים
|
|  +-- UserControls/         <- פקדים (לוח, כרטיס שחקן)
|
|  +-- Converters/, Validation/ <- ממירים וחוקי בדיקה
|
|  +-- Services/CheckersClient.cs <- WCF עוטף
|
+-- WebApplication1/         <- אתר ניהול (ASP.NET Razor)
|
|  +-- Pages/Admin/          <- Dashboard, Users, Games
|
|  +-- Services/             <- AdminAuth, CheckersBackend
|
+-- mobile/CheckersMaui/     <- MAUI אפליקציה
    +-- Pages/               <- Home, Login, Profile, ...
    +-- Services/AppState, ServiceHelper

```

5.3 תרשים פריסה (Deployment Diagram)

מבחינת פריסה, המערכת רצה בכמה צמתים:

מחשב השרת: מארח את תהליך שרת ה-WCF המאזין על נקודות הקצה net.tcp (פורט 9100) ו-basicHttp (פורט 8733), וכן את קובץ ה-Access.

אתר הניהול: שרת ASP.NET Core המתחבר לשרת ה-WCF ומציג את פאנל הניהול בדפדפן.

מחשבי לקוח (WPF Desktop): מותקנים על מחשבי המשתמשים, ומתחברים לשרת ה-WCF.

מכשירי מובייל (MAUI): מתחברים לשרת דרך הרשת.

5.4 זרימת הנתונים בין הרכיבים

נדגים את זרימת המידע בשני תרחישים:

תרחיש 1, ביצוע מהלך במשחק רב-משתתפים:

השחקן בוחר כלי ויעד בלקוח ה-WPF ולוחץ לביצוע.

הלקוח שולח את מסלול המהלך לשרת באמצעות הפעולה MakeMove.

השרת מעביר את המהלך דרך CheckersEngine לאימות; אם אינו חוקי הוא נדחה.

אם המהלך חוקי, השרת מעדכן את מצב הלוח, מעביר את התור, ושומר את המהלך בטבלת Moves.

שני הלקוחות, הדוגמים את מצב המשחק בלולאת Polling, מקבלים את המצב המעודכן ומציגים אותו.

תרחיש 2, פתיחת טבלת מובילים מ-MAUI:

המשתמש לוחץ על מסך טבלת המובילים באפליקציה.

האפליקציה קוראת לפעולה GetTopPlayers בשרת.

השרת קורא ל-UsersDB.GetTopPlayers שמבצע שאילתת `SELECT TOP 50 ORDER BY Rating`.

התוצאה מוחזרת לאפליקציה שמציגה אותה ברשימה.

6. צד שרת

6.1 רכיבי השרת

המערכת מכילה שני רכיבי שרת, כל אחד עם תחום אחריות:

שרת (server) WCF: השרת המרכזי, המשרת את לקוח ה-WPF ואת אפליקציית ה-MAUI. הוא מבוסס net.tcp לתקשורת מהירה ו-basicHttp לתאימות רחבה, ומתארח כתהליך Console עצמאי שעולה, מאתחל את מסד הנתונים, ויוצר חשבון מנהל ברירת מחדל אם אינו קיים.

אתר ניהול (WebApplication1): שרת ASP.NET Core המציג את פאנל הניהול בדפדפן ומתחבר לשרת ה-WCF דרך לקוח שנוצר אוטומטית.

שני הרכיבים חולקים את אותו מסד נתונים, כך שמשמש שנרשם דרך לקוח אחד יכול להתחבר מכל לקוח אחר עם אותם פרטים.

6.2 שירות WCF ראשי (ICheckersService)

החוזה של ה-WCF מוגדר בממשק ICheckersService, וכל פעולה מעוטרת ב-[OperationContract]. הממשק מציג את כל הפעולות שהלקוח יכול לקרוא להן, מחולקות לקבוצות:

אימות: Login (מחזירה טוקן), Register, ו-Logout.

התאמת יריבים: JoinQueue, LeaveQueue, ו-JoinBotGame.

מצב המשחק: GetLegalMovesForSquare, GetCurrentGame, ו-GetMoveHistory.

פעולות במשחק: RespondToDraw, MakeMove, Resign, OfferDraw, ו-RespondToDraw.

פרופילים: UpdateMyProfilePicture, GetUserProfile, UpdateMyEmail, ChangeMyPassword, ו-UpdateMyProfilePicture.

סטטיסטיקות: GetOnlinePlayers, GetTopPlayers, GetMyGameHistory, ו-GetOnlinePlayers.

ניהול: AdminGetServerStats, AdminGetAllUsers, AdminBanUser, AdminGetRecentGames, ו-AdminGetServerStats.

האימות מבוסס טוקן: הפעולה Login מחזירה מזהה ייחודי (Guid) שאותו הלקוח שולח בכל קריאה נוספת, וכך השרת מזהה מי הקורא.

6.3 לוגיקה עסקית (GameManager ו-GameSession)

הלוגיקה של ניהול המשחקים מרוכזת במחלקת GameManager, ומצב של משחק יחיד נשמר במחלקת GameSession.

ניהול state בזיכרון: כל משחק פעיל נשמר בזיכרון השרת. כך השרת מטפל בכמה משחקים בו-זמנית מבלי לגעת במסד הנתונים בכל מהלך; רק בסיום המשחק התוצאה נשמרת ב-DB.

התאמת יריבים (Matchmaking): כשמישהו מבקש משחק, השרת בודק אם יש שחקן אחר הממתין בתור (עם שם משתמש שונה). אם כן נוצר משחק חדש; אחרת השחקן נכנס לתור עד שיגיע יריב.

סנכרון תורות: מצב התור נשמר ב-GameSession. מהלך מאושר רק כשהוא תורו של השחקן ורק לאחר אימות חוקיותו, וכך נמנע מצב שבו שני שחקנים פועלים בו-זמנית.

6.4 תכנות אסינכרוני (Asynchronous Programming)

הפעולות מול מסד הנתונים והקריאות לשרת מבוצעות באופן אסינכרוני, כדי שהממשק לא ייתקע בזמן ההמתנה לתשובה.

בצד הלקוח: כל קריאה לשרת מבוצעת על Thread נפרד, וכל עדכון ממשק מתבצע בחזרה על ה-UI Thread.

בצד השרת: שכבת הגישה לנתונים משתמשת בפעולות אסינכרוניות, כך שבעוד Thread אחד ממתין לתשובת מסד הנתונים, השרת ממשיך לטפל בבקשות של שחקנים אחרים. דוגמה לחתימה אסינכרונית:

```
public async Task<User> GetByUsernameAsync(string username)
```

6.5 שכבת גישה הנתונים (Data Access Layer)

שכבת ה-DB מורכבת ממחלקה אבסטרקטית BaseDB וממחלקות יורשות UsersDB, GamesDB, MovesDB. הירושה מאפשרת שיתוף קוד (פעולות SELECT, INSERT, UPDATE, DELETE גנריות), ובזמנית מאפשרת לכל יורשת לממש את אופן ההמרה של רשומה לאובייקט מודל.

BaseDB

המחלקה האבסטרקטית מספקת: מחרוזת חיבור סטטית הנבנית פעם אחת, אובייקט OleDb לחיבור ולפקודות, פעולה אבסטרקטית ליצירת אובייקט ריק, פעולה וירטואלית להמרת רשומה לאובייקט, פעולת SELECT גנרית המחזירה רשימה באמצעות פולימורפיזם, ופעולות עדכון גנריות. כל הפעולות תומכות בפרמטרים מוגנים מפני SQL Injection.

UsersDB

מוסיפה פעולות לשליפת משתמש לפי שם (להתחברות), שליפת כל המשתמשים (לפאנל הניהול), עדכון פרופיל וסיסמה, שליפת טבלת המובילים, וחסימת משתמש.

GamesDB

מטפלת בשמירת משחקים שהסתיימו ובשליפת היסטוריית המשחקים של שחקן, כולל שאילתת חיבור (Join) מול טבלת המשתמשים לשליפת שמות שני השחקנים בבקשה אחת.

MovesDB

מטפלת בשמירת כל מהלך ובשליפת רצף המהלכים המלא של משחק, לצורך הצגת צפייה חוזרת.

6.6 אתר הניהול (ASP.NET Razor Pages)

אתר הניהול הוא רכיב שרת נפרד המספק למנהל ממשק בדפדפן. הוא בנוי ב-ASP.NET Core Razor Pages ומתחבר לשרת ה-WCF דרך לקוח שנוצר אוטומטית. האתר כולל לוח בקרה עם נתוני השרת, מסך ניהול משתמשים (כולל חסימה), ומסך צפייה במשחקים אחרונים. הגישה אליו מוגנת באמצעות עוגיית התחברות (Cookie Authentication) ובדיקת הרשאת מנהל.

6.7 תקשורת בזמן אמת (Polling)

בשונה ממערכות המבוססות דחיפה (Push), עדכון מצב המשחק בלקוחות מבוסס דגימה (Polling): לקוח ה-WPF ולקוח ה-MAUI שואלים את השרת על מצב המשחק במרווחי זמן קצרים (כחצי שנייה), ומציגים את המצב המעודכן. גישה זו פשוטה ליישום, אינה דורשת חיבור קבוע, ומתאימה לקצב של משחק תורות.

7. צד לקוח

7.1 לקוח WPF Desktop

עיצוב הממשק: לקוח ה-WPF מעוצב בערכת צבעים אחידה ונקייה, עם סגנונות גלובליים המוגדרים בקובץ App.xaml ומאפשרים שימוש חוזר ואחידות ויזואלית בכל המסכים.

דפי המערכת:

LoginView: התחברות ורישום, עם שדות קלט ובדיקות תקינות (Validation), ופקד ייעודי להצגת הסיסמה או הסתרתה.

LobbyView: המסך המרכזי, המציג את פרטי השחקן, את רשימת המחברים ואת היסטוריית המשחקים, ומאפשר התחלת חיפוש משחק.

GameView: לוח המשחק, המציג לוח 8x8, את הכלים, את שמות שני השחקנים, תמונות הפרופיל והדירוג, ומאפשר ביצוע מהלכים, הצעת תיקו והתפטרות.

GameOverView, ProfileView, ReplayView, AdminView: מסכי סיום, פרופיל, צפייה חוזרת וניהול.

ניהול ה-WCF וה-MVVM: המחלקה CheckersClient עוטפת את הקריאות לשרת ומחזיקה את הטוקן. הלקוח בנוי בתבנית MVVM: כל מסך מנוהל על-ידי ViewModel המחזיק את הנתונים ואת הפקודות (RelayCommand), ויורש ממחלקת בסיס המיישמת עדכון ממשק אוטומטי. ה-ViewModel של מסך המשחק דוגם את מצב המשחק מהשרת במרווחים קצרים ומעדכן את הלוח.

7.2 לקוח אתר הניהול (Web)

אתר הניהול הוא לקוח מבוסס דפדפן הבנוי ב-ASP.NET Razor Pages. כל דף מורכב מקובץ תצוגה (cshtml) ומקובץ לוגיקה (PageModel). האתר מספק למנהל לוח בקרה עם נתוני השרת, מסך ניהול משתמשים עם אפשרות חסימה, ומסך צפייה במשחקים אחרונים. הנתונים מתרעננים אוטומטית, והגישה מוגבלת למשתמשים בעלי הרשאת מנהל בלבד.

7.3 לקוח MAUI (Android ו-Windows)

ארכיטקטורה: .NET MAUI מאפשר לכתוב פרויקט #C יחיד שמתקמפל לאפליקציות נטיביות לכמה פלטפורמות. האפליקציה מכוונת בעיקר ל-Android, וגם ל-Windows לצורך פיתוח ובדיקה מהירים.

שירותים מרכזיים:

AppState: שומר את כתובת השרת, הטוקן ושם המשתמש המחובר.

ServiceHelper: מספק גישה נוחה לשירותים המשותפים של האפליקציה.

דפי הלקוח: HomePage (מסך הבית), LoginPage ו-SignUpPage (אימות), ProfilePage (פרופיל), LeaderboardPage (טבלת מובילים), ו-GameHistoryPage (היסטוריית משחקים).

עיצוב אחיד: שלושת הלקוחות חולקים את אותה זרימת מסכים ואת אותו היגיון ויזואלי, כך שמשתמש שמכיר אחד מהם יכול להשתמש בקלות באחר.

8. אבטחה והגנה על נתונים

8.1 הצפנת סיסמאות עם PBKDF2

סיסמאות הן הנתון הרגיש ביותר במערכת. אסור לאחסן אותן בטקסט גלוי, ואסור גם להצפין אותן באלגוריתם הפיד, כי אם מסד הנתונים נפרץ, מישוהו יוכל לפענח אותן בחזרה.

הפתרון הנכון הוא תמצית (Hash) חד-כיוונית, פונקציה שלא ניתן להפוך. במערכת השתמשתי ב-PBKDF2 עם פונקציית הגיבוב SHA-256. לכל סיסמה נוצר Salt אקראי, והתמצית מחושבת במספר רב של סבבים, מה שמקשה מאוד על ניסיונות פיצוח.

```
public static string Hash(string password)
{
    byte[] salt = RandomNumberGenerator.GetBytes(16);
    var pbkdf2 = new Rfc2898DeriveBytes(password, salt, 60000, HashAlgorithmName.SHA256);
    byte[] hash = pbkdf2.GetBytes(32);
    return $"{60000}.{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
}
```

התמצית נשמרת בפורמט הכולל את מספר הסבבים, ה-Salt וה-Hash. בעת ההתחברות מחושבת תמצית הסיסמה שהוזנה ומושווית לתמצית השמורה בהשוואה בזמן קבוע (Constant-Time Comparison), מה שמונע התקפות תזמון (Timing Attacks).

8.2 הגנה מ-SQL Injection

SQL Injection היא אחת ההתקפות הנפוצות על מסדי נתונים. היא מתרחשת כשמכניסים קלט משתמש ישירות לתוך מחרוזת SQL, ומשתמש זדוני יכול להזריק תוכן שמשנה את משמעות השאילתה.

ההגנה היחידה האמיתית היא שימוש בפרמטרים (Parameterized Queries). כל שאילתה במערכת משתמשת בפרמטר OleDb עם סימן ? :

```
// SQL Injection - נכון, מוגן מ
string sql = "SELECT * FROM Users WHERE Username = ?";
command.Parameters.AddWithValue("?", username);

// SQL Injection - שגוי, פגיע ל
// string sql = "SELECT * FROM Users WHERE Username = '" + username + "'";
```

הספק של OleDb מתייחס לערך כפרמטר ולא כחלק מתחביר ה-SQL, ולכן גם אם המשתמש מזין תוכן זדוני, הוא נבדק כשם משתמש בלבד ואינו משנה את השאילתה.

8.3 ניהול אימות מבוסס טוקן

האימות במערכת מבוסס טוקן : בעת התחברות מוצלחת מייצר השרת מזהה ייחודי (Guid) ושומר אותו יחד עם שם המשתמש המשוך. הלקוח מקבל את הטוקן ושולח אותו בכל קריאה נוספת. השרת מאמת את הטוקן בכל פעולה, מאתר את המשתמש המשוך, ורק אז מבצע את הבקשה. טוקן שאינו תקין נדחה מיד. גישה זו שומרת על השרת חסר-מצב מבחינת זהות הלקוח ומאפשרת ניתוק נקי בעת Logout.

8.4 אכיפת חוקים בצד השרת

שכבת אבטחה חשובה נוספת היא אכיפת כל לוגיקת המשחק בצד השרת בלבד. הלקוח מציג את הלוח ושולח את המהלך המבוקש, אך תקינות המהלך נבדקת אך ורק בשרת על-ידי CheckersEngine. כך שחקן אינו יכול לזייף אכילה, לדלג על אכילת חובה, להזיז כלי של היריב או לבצע מהלך לא חוקי, גם אם ישנה את קוד הלקוח.

8.5 הגנת Admin

פאנל הניהול מוגן בשתי שכבות: בצד הלקוח, מסך הניהול נחשף רק למשתמש שהרשאתו Admin ; ובצד השרת, פעולות הניהול (כגון חסימת משתמש) מאמתות את הטוקן ובודקות שלמשתמש המבצע יש הרשאת מנהל לפני הביצוע. כך לא ניתן לעקוף את ההגנה דרך פנייה ישירה לשרת.

9. תהליכים מרכזיים במערכת

9.1 תהליך התחברות

התחברות היא התהליך הראשון שכל משתמש עובר. השלבים :
המשתמש מזין שם משתמש וסיסמה במסך ההתחברות.
הלקוח שולח את הפרטים לשרת באמצעות הפעולה Login.
השרת קורא ל-UsersDB לשליפת המשתמש לפי שם, באמצעות שאילתה עם פרמטר.
אם המשתמש קיים, השרת מאמת את הסיסמה באמצעות PasswordHasher (PBKDF2).
אם הסיסמה תקינה, השרת מייצר טוקן (Guid), שומר אותו, ומחזיר אותו ללקוח.
הלקוח שומר את הטוקן ועובר ללובי.

9.2 תהליך התאמת משחק (Matchmaking)

בלחיצה על חיפוש משחק, מנהל ההתאמה בשרת מנהל את התור :
הלקוח קורא לפעולה JoinQueue.
השרת בודק אם כבר ממתין שחקן אחר בתור (עם שם שונה).
אם כן, שני השחקנים מוצאים מהתור, נוצר משחק חדש, ושני הצדדים מקבלים הודעה שהמשחק התחיל.
אם לא, השחקן מצטרף לתור וממתין.
בכל רגע השחקן יכול לבטל את החיפוש באמצעות LeaveQueue.

9.3 תהליך פתיחת משחק

בשונה ממשחקים שדורשים הצבת כלים, בדמקה הכלים ניצבים במצב פתיחה קבוע. לאחר שנמצא יריב (או מיד, אם נבחר משחק מול הבוט):
השרת יוצר מצב לוח התחלתי עם הכלים בעמדותיהם.
השרת מגריל מי מתחיל ושולח לשני הצדדים את מצב הלוח הראשוני.
שני הצדדים מועברים למסך המשחק, והשחקן שתורו ראשון מתחיל.

9.4 תהליך ביצוע מהלך

ביצוע מהלך הוא הפעולה המרכזית במשחק. כאשר השחקן בוחר כלי ויעד ולוחץ לביצוע :
הלקוח קורא לפעולה MakeMove עם מסלול המהלך (משבצת מקור ומשבצות היעד).
השרת מוודא שזהו תורו של השחקן.
השרת מעביר את המסלול ל-CheckersEngine, המחשב את כל המהלכים החוקיים ובודק אם המהלך שנשלח נמצא ביניהם.
אם המהלך אינו חוקי, הוא נדחה והלוח אינו משתנה.
אם המהלך חוקי, השרת מחיל אותו על הלוח: מזיז את הכלי, מסיר את הכלים שנאכלו לאורך המסלול, ומבצע הכתרה אם הכלי הגיע לשורה האחורית.
השרת מעביר את התור לצד השני, שומר את המהלך בטבלת Moves, ובודק אם המשחק הסתיים (אם לאחד הצדדים אין כלים או אין מהלך חוקי).

אם המשחק הסתיים, השרת מעדכן את התוצאה, מחשב את שינוי ה-Elo, ושומר את המשחק.

9.5 אלגוריתם היריב הממוחשב (Bot)

היריב הממוחשב מיושם במחלקת BotEngine. בתורו הוא פונה ל-CheckersEngine לקבלת רשימת כל המהלכים החוקיים שלו, ובוחר מתוכם מהלך. הבחירה אינה אקראית לחלוטין: הבוט מעדיף מהלכים המובילים להכתרת מלך, ובכך משחק בצורה מעט חכמה יותר. מכיוון שכל המהלכים מגיעים מאותו מנוע חוקים שמשמש את השחקנים האנושיים, הבוט לעולם אינו מבצע מהלך לא חוקי ומכבד את חובת האכילה.

9.6 תהליך הניתוק

ניתוק יכול לקרות בכמה אופנים:

ניתוק יזום: המשתמש לוחץ על התפטרות (Resign). הצד השני מקבל הודעת ניצחון, והתוצאה נשמרת.

סגירת חלון: בלקוח ה-WPF, בעת סגירת החלון נשלחת הודעת עזיבה לשרת לפני הסגירה.

ניתוק רשת או קריסה: אם לקוח מפסיק להגיב, טיימר ה-StaleSweep בשרת מזהה זאת לאחר פרק זמן מוגדר, מסיים את המשחק לטובת היריב, ומטיל קנס דירוג על הנוטש.

10. בדיקות ותסריטי שימוש

10.1 בדיקות ידניות שביצעת

במהלך הפיתוח ביצעתי בדיקות ידניות שיטתיות לכל תרחיש. הבדיקות חולקו לקטגוריות:

בדיקות אימות וגישה: רישום עם פרטים תקינים ולא תקינים, התחברות עם סיסמה נכונה ושגויה, ניסיון גישה לפאנל ניהול ללא הרשאה.

בדיקות מנוע המשחק: ביצוע מהלך פשוט, ניסיון מהלך לא חוקי, אכילה רגילה, אכילה כפולה, אכילת חובה, והכתרת מלך.

בדיקות משחק רב-משתתפים: התאמת שני שחקנים, ביצוע מהלכים לסירוגין, הצעת תיקו והתפטרות, וסיום משחק עם עדכון דירוג.

בדיקות פאנל ניהול: צפייה ברשימת המשתמשים, חסימת משתמש, וצפייה במשחקים אחרונים.

10.2 בדיקות עומס

בנוסף לבדיקות הפונקציונליות, בחנתי כמה משחקים יכולים לרוץ בו-זמנית:

כמה משחקים פעילים במקביל רצו ללא בעיות, והשרת הגיב בזמן.

עם עליית מספר המשחקים הפעילים השרת נשאר יציב, מכיוון שכל משחק מנוהל בזיכרון ורק התוצאה הסופית נכתבת למסד הנתונים.

הדגימה התכופה (Polling) מצד הלקוחות נבדקה כדי לוודא שהיא אינה מעמיסה יתר על המידה על השרת.

10.3 בדיקות אבטחה

בדקתי גם את ההגנות של המערכת: ניסיתי להזין תוכן זדוני בשדות הקלט כדי לוודא שההגנה מפני SQL Injection פועלת; ניסיתי לבצע מהלך לא חוקי ישירות מול השרת כדי לוודא שהוא נדחה; ובדקתי שסיסמאות נשמרות מוצפנות בלבד במסד הנתונים ולא בטקסט גלוי.

11. מדריך למשתמש

11.1 איך מתחילים?

כדי לשחק ב-Online Checkers ניתן לבחור באחת מהדרכים הבאות:
הפעלת אפליקציית ה-WPF מהמחשב האישי.
כניסה לאתר הניהול בדפדפן (למנהלים).
התקנת אפליקציית ה-MAUI על מכשיר אנדרואיד.

11.2 רישום משתמש חדש

בפתיחה הראשונה:
לוחצים על "רישום".
ממלאים שם משתמש ייחודי, סיסמה, ודוא"ל.
לוחצים על כפתור הרישום. אם הכול תקין, מתבצע מעבר למסך ההתחברות.

11.3 התחברות

עם הפרטים מהרישום:
מזינים שם משתמש וסיסמה.
לוחצים על כפתור ההתחברות.
אם הכול תקין, מתבצע מעבר ללובי; אחרת מוצגת הודעת שגיאה.

11.4 תפריט ראשי, מה ניתן לעשות?

מהלובי ניתן לבצע את הפעולות הבאות:
חיפוש משחק: כניסה לתור התאמה מול שחקן אחר.
משחק מול הבוט: התחלת משחק מול היריב הממוחשב.
טבלת מובילים: צפייה בדירוג הגלובלי של כל השחקנים.
היסטוריה: צפייה ברשימת המשחקים שלי וצפייה חוזרת בהם.
פרופיל: עדכון דוא"ל, סיסמה ותמונת פרופיל.
יציאה: התנתקות מהמערכת.

11.5 ביצוע מהלכים

במהלך המשחק, בתורך:
לוחצים על כלי שלך. המשבצות שאליהן ניתן לנוע מודגשות.
לוחצים על משבצת היעד כדי לבצע את המהלך.
אם קיימת אכילת חובה, ניתן לבצע רק מהלכי אכילה.
במקרה של שרשור אכילות, ממשיכים לבחור את משבצות הנחיתה עד לסיום הרצף.

11.6 המשחק

מסך המשחק מציג את לוח 8×8 עם הכלים של שני הצדדים, את שמות שני השחקנים, תמונות הפרופיל והדירוג שלהם. בתחתית המסך נמצאים כפתורים להצעת תיקו ולהתפטרות. בכל תור מודגש הכלי הנבחר והמהלכים החוקיים שלו, והשרת מעדכן את הלוח לאחר כל מהלך של אחד הצדדים.

11.7 רוצה לשחק מול חבר או מול הבוט?

כדי לשחק מול חבר, שניכם נכנסים למערכת ולוחצים על חיפוש משחק, והשרת מתאים ביניכם אוטומטית. אם אין יריב פנוי או שרוצים לתרגל לבד, ניתן לבחור משחק מול הבוט, שמתחיל מיד.

12. סיכום אישי, רפלקציה

הפרויקט הזה היה חוויה גדולה. התחלתי עם רעיון פשוט, משחק דמקה מקוון, וגיליתי שמתחת לזה מסתתר עולם שלם של נושאים שלא הכרתי. הקושי האמיתי לא היה הקוד עצמו, אלא לחזור אליו שוב ושוב אחרי שמשהו לא עבד, ולא לוותר עד שמצאתי את הפתרון.

למדתי לעומק כיצד בונים ארכיטקטורת שרת-לקוח, מדוע חשוב להפריד בין שכבות, ולמה כדאי לאכוף לוגיקה רגישה בצד השרת בלבד. למדתי על מסדי נתונים ושאלתות מאובטחות, על תכנות אסינכרוני, ועל אבטחת מידע מעשית כמו הצפנת סיסמאות והגנה מפני SQL Injection. נהניתי במיוחד לראות את כל החלקים, WPF, WCF, אתר הניהול ו-MAUI, מתחברים יחד למערכת אחת שעובדת. זה היה הפרויקט הגדול והמאתגר ביותר שעשיתי, ואני יוצא ממנו מבין הרבה יותר ממה שהבנתי בתחילתו.

13. ביבליוגרפיה

13.1 ספרות ומדריכי לימוד

התיעוד הרשמי של Microsoft לכל הטכנולוגיות (WPF, WCF, ASP.NET Core, MAUI): <https://learn.microsoft.com>

חוברת הלימוד של חלופת "שירותי אינטרנט, תכנות אסינכרוני ומסדי נתונים" של משרד החינוך.
חומרי הלימוד וההנחיות של המורה המנחה.

13.2 אתרי עזר טכניים

Stack Overflow: לפתרון בעיות ספציפיות, במיוחד בנושאי WCF ו-OleDb.
Microsoft Q&A: לשאלות מתקדמות על MAUI ועל WPF.
GitHub: לקריאת קוד פתוח לדוגמה לצורך לימוד בלבד.

13.3 כלים וספריות צד שלישי

System.Data.OleDb: ספריית הגישה למסד הנתונים Microsoft Access.
רכיבי ההצפנה המובנים של .NET (PBKDF2 דרך Rfc2898DeriveBytes) להצפנת סיסמאות.
WCF ו-ASP.NET Core: התשתיות של .NET. לבניית השרת ואתר הניהול.

13.4 שימוש בבינה מלאכותית

בפיתוח הפרויקט נעזרתי גם בכלי בינה מלאכותית. השימוש היה כדלקמן:
הסבר על מושגים שלא הבנתי לעומק.
דיון על בחירות ארכיטקטוניות, כגון אופן חיבור הלקוחות לשרת.
עזרה באיתור באגים מורכבים.
ניסוח מסמכים, כמו ספר הפרויקט הזה, על בסיס נקודות שכתבתי בעצמי.
השתדלתי להבין כל שורת קוד שמופיעה בפרויקט, ובמיוחד בקטעים המורכבים כמו אלגוריתם יצירת המהלכים והאכילה הכפויה התעכבתי ולמדתי לעומק לפני שהמשכתי.

14. נספחים, קוד מלא

14.1 מבנה הפרויקטים בסולושן

הפרויקט מורכב משרת WCF (server), לקוח WPF (client), אתר ניהול (WebApplication1), ואפליקציית MAUI (mobile/CheckersMaui). השרת מחולק לתת-שכבות: Model, BusinessLogic ו-ViewDB. הלקוחות מתחברים כולם לאותו שירות דרך חוזה ה-DataContract המשותף. המבנה המלא מוצג בסעיף 5.2.

14.2 דוגמת קוד, מחלקת BaseDB (מחלקה אבסטרקטית)

המחלקה האבסטרקטית BaseDB מרכזת את הגישה למסד הנתונים. כל מחלקות ה-DB יורשות ממנה. להלן השלד שלה:

```
public abstract class BaseDB
{
    protected static string connectionString;
    protected OleDbConnection connection;
    protected OleDbCommand command;
    protected OleDbDataReader reader;

    // כל יורשת מחזירה אובייקט ריק מהסוג שלה
    protected abstract Base NewEntity();

    // לאובייקט מודל reader-כל יורשת ממירה רשומה מה
    protected virtual void CreateModel(Base entity) { /* ... */ }

    // גנרית, מחזירה רשימה באמצעות פולימורפיזם SELECT פעולת
    protected List<Base> Select(string sql)
    {
        var list = new List<Base>();

        // פתיחת חיבור, הרצת הפקודה, קריאת שורה שורה
        // וסגירת החיבור, CreateModel ו- NewEntity עם
        return list;
    }

    // (SQL Injection-מוגן מ) פעולת עדכון גנרית עם פרמטרים
    protected int SaveChanges(OleDbCommand cmd) { /* ... */ }
}
```

code

code

code

14.3 דוגמת קוד, מנוע המשחק (יצירת מהלכים)

הפעולה המרכזית במנוע היא יצירת כל המהלכים החוקיים של הצד התורן, תוך כיבוד חובת האכילה:

```

public static List<Move> GenerateLegalMoves(Board board)
{
    var jumps = new List<Move>();
    var slides = new List<Move>();

    // סריקת כל הכלים של הצד התורן
    for (int f = 0; f < 8; f++)
        for (int r = 0; r < 8; r++)
        {
            Piece p = board[f, r];

            if (p == null || p.Color != board.SideToMove) continue;

            GenerateJumpMovesFrom(board, new Square(f, r), p, jumps);

            if (jumps.Count == 0)
                GenerateSlideMovesFrom(board, new Square(f, r), p, slides);
        }

    // חובת אכילה: אם קיימת אכילה, רק היא חוקית
    return jumps.Count > 0 ? jumps : slides;
}

```

14.4 דוגמת קוד, הצפנת סיסמאות (PasswordHasher)

מחלקת ההצפנה משתמשת ב-PBKDF2 עם SHA-256, Salt ואקראי, ומספר רב של סבבים:

```

public static string Hash(string password)
{
    byte[] salt = RandomNumberGenerator.GetBytes(16);
    var k = new Rfc2898DeriveBytes(password, salt, 60000, HashAlgorithmName.SHA256);
    byte[] hash = k.GetBytes(32);
    return $"{60000}.{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
}

```

code

```

public static bool Verify(string password, string stored)
{
    string[] parts = stored.Split('.');
    int iters = int.Parse(parts[0]);
    byte[] salt = Convert.FromBase64String(parts[1]);
    byte[] hash = Convert.FromBase64String(parts[2]);
    var k = new Rfc2898DeriveBytes(password, salt, iters, HashAlgorithmName.SHA256);
    byte[] test = k.GetBytes(32);
    return CryptographicOperations.FixedTimeEquals(test, hash);
}

```

}

14.5 הערה לסיום

כל קטעי הקוד שהוצגו בנספחים הם תמצית של הקוד האמיתי במערכת, שנועדה להמחיש את העקרונות המרכזיים. הקוד המלא מאורגן בפרויקטים שתוארו לאורך הספר, תוך שמירה על הפרדה בין השכבות ועל אכיפת כל לוגיקת המשחק בצד השרת.